Embedded Systems Lab Programs

Labs 1-5 contain corrected ARM assembly listings; build each as a separate Keil ARMasm/Cortex-M3 program without a second startup vector table. The five hardware C listings at the end are reproduced from the uploaded code without corrections. Keil compilation was not available for verification.

Lab 1

2 programs

Data Transfer

1. Transfer a 32-bit value between data memory locations

Exercise file: transfer data memory.txt

Copies a word between two allocated data-memory locations.

```
        PRESERVE8
        AREA RESET, DATA, READONLY
        EXPORT __Vectors
__Vectors
        DCD 0x10001000
        DCD Reset_Handler
        ALIGN
        AREA mycode, CODE, READONLY
        THUMB
        ENTRY
        EXPORT Reset_Handler
Reset_Handler
        LDR R0, =SOURCE
        LDR R1, =DEST
        LDR R2, [R0]
        STR R2, [R1]
STOP    B STOP
        AREA mydata, DATA, READWRITE
SOURCE  DCD 0x12345678
DEST    DCD 0
        END
```

2. Swap the contents of two registers

Exercise file: swap contents.txt

Uses a temporary register, arithmetic, and XOR, storing each swapped pair.

```
        PRESERVE8
        AREA RESET, DATA, READONLY
        EXPORT __Vectors
__Vectors
        DCD 0x10001000
        DCD Reset_Handler
        ALIGN
        AREA mycode, CODE, READONLY
        THUMB
        ENTRY
        EXPORT Reset_Handler
Reset_Handler
        LDR R0, =0x00011234
        LDR R1, =0x00002123
        MOV R2, R1
        MOV R1, R0
        MOV R0, R2
        LDR R3, =TEMP_PAIR
        STR R0, [R3]
        STR R1, [R3, #4]

        LDR R0, =0x00011234
        LDR R1, =0x00002123
        ADD R0, R0, R1
        SUB R1, R0, R1
        SUB R0, R0, R1
        LDR R3, =ARITH_PAIR
```

```
        STR R0, [R3]
        STR R1, [R3, #4]

        LDR R0, =0x00011234
        LDR R1, =0x00002123
        EOR R0, R0, R1
        EOR R1, R0, R1
        EOR R0, R0, R1
        LDR R3, =XOR_PAIR
        STR R0, [R3]
        STR R1, [R3, #4]
STOP    B STOP
        AREA mydata, DATA, READWRITE
TEMP_PAIR  DCD 0, 0
ARITH_PAIR DCD 0, 0
XOR_PAIR   DCD 0, 0
        END
```

Lab 2

3 programs

Arithmetic Instructions

1. Add two values in registers

Exercise file: ADD.txt

Places two immediate values in registers and adds them.

```
        PRESERVE8
        AREA RESET, DATA, READONLY
        EXPORT __Vectors
__Vectors
        DCD 0x10001000
        DCD Reset_Handler
        ALIGN
        AREA mycode, CODE, READONLY
        THUMB
        ENTRY
        EXPORT Reset_Handler
Reset_Handler
        MOV R0, #3
        MOV R1, #10
        ADD R0, R0, R1
        LDR R2, =RESULT
        STR R0, [R2]
STOP    B STOP
        AREA mydata, DATA, READWRITE
RESULT DCD 0
        END
```

2. Add two 32-bit numbers

Exercise file: add 32 bit numbers.txt

Reads two values from code memory, adds them, and stores the result in data memory.

```
        PRESERVE8
        AREA RESET, DATA, READONLY
        EXPORT __Vectors
__Vectors
        DCD 0x10001000
        DCD Reset_Handler
        ALIGN
        AREA mycode, CODE, READONLY
        THUMB
        ENTRY
        EXPORT Reset_Handler
Reset_Handler
        LDR R0, =VALUE1
        LDR R1, [R0]
        LDR R0, =VALUE2
        LDR R3, [R0]
        ADDS R6, R1, R3
```

```
        LDR R2, =RESULT
        STR R6, [R2]
STOP    B STOP
VALUE1  DCD 0x12345678
VALUE2  DCD 0xABCDEF12
        AREA mydata, DATA, READWRITE
RESULT DCD 0
        END
```

3. Add and subtract 64-bit values

Exercise file: add and sub 2 64 bits.txt

Computes a two-word sum and difference, propagating carry or borrow into the high word.

```
        PRESERVE8
        AREA RESET, DATA, READONLY
        EXPORT __Vectors
__Vectors
        DCD 0x10001000
        DCD Reset_Handler
        ALIGN
        AREA mycode, CODE, READONLY
        THUMB
        ENTRY
        EXPORT Reset_Handler
Reset_Handler
        LDR R0, =0xF2345678    ; A low word
        LDR R1, =0x00000012    ; A high word
        LDR R2, =0x20000001    ; B low word
        LDR R3, =0x00000022    ; B high word
        ADDS R4, R0, R2       ; sum low, set carry
        ADC R5, R1, R3        ; sum high plus carry
        SUBS R6, R0, R2       ; difference low, set borrow
        SBC R7, R1, R3        ; difference high minus borrow
        LDR R8, =RESULT
        STR R4, [R8]
        STR R5, [R8, #4]
        STR R6, [R8, #8]
        STR R7, [R8, #12]
STOP    B STOP
        AREA mydata, DATA, READWRITE
RESULT  DCD 0, 0, 0, 0       ; sum low/high, difference low/high
        END
```

Lab 3

12 programs

Arithmetic and Logical Instructions

1. Multiply two 32-bit numbers

Exercise file: mul of 2 32 bits.txt

Multiplies two values held in registers.

```
        PRESERVE8
        AREA RESET, DATA, READONLY
        EXPORT __Vectors
__Vectors
        DCD 0x10001000
        DCD Reset_Handler
        ALIGN
        AREA mycode, CODE, READONLY
        THUMB
        ENTRY
        EXPORT Reset_Handler
Reset_Handler
        LDR R0, =0x00000123
        LDR R1, =0x00000003
        MUL R2, R0, R1
        LDR R3, =RESULT
        STR R2, [R3]
STOP    B STOP
```

```
        AREA mydata, DATA, READWRITE
RESULT DCD 0
        END
```

2. Multiply two 64-bit numbers

Exercise file: mul of 2 64 bits.txt

Combines partial products to calculate the lower 64 bits of the result.

```
        PRESERVE8
        AREA RESET, DATA, READONLY
        EXPORT __Vectors
__Vectors
        DCD 0x10001000
        DCD Reset_Handler
        ALIGN
        AREA mycode, CODE, READONLY
        THUMB
        ENTRY
        EXPORT Reset_Handler
Reset_Handler
        LDR R0, =0x00000005    ; A low word
        LDR R1, =0x00000000    ; A high word
        LDR R2, =0x00000005    ; B low word
        LDR R3, =0x00000000    ; B high word
        UMULL R4, R5, R0, R2  ; low-word product
        MUL R6, R0, R3        ; A.low * B.high
        MLA R6, R1, R2, R6   ; plus A.high * B.low
        ADD R5, R5, R6        ; low 64 bits of product
        LDR R7, =RESULT
        STR R4, [R7]
        STR R5, [R7, #4]
STOP    B STOP
        AREA mydata, DATA, READWRITE
RESULT DCD 0, 0
        END
```

3. Multiply values in a separate routine

Exercise file: mul.txt

Calls a multiplication subroutine and stores its 32-bit result.

```
        PRESERVE8
        AREA RESET, DATA, READONLY
        EXPORT __Vectors
__Vectors
        DCD 0x10001000
        DCD Reset_Handler
        ALIGN
        AREA mycode, CODE, READONLY
        THUMB
        ENTRY
        EXPORT Reset_Handler
Reset_Handler
        BL MULTIPLY
        LDR R3, =RESULT
        STR R2, [R3]
STOP    B STOP

MULTIPLY
        LDR R0, =0x706F
        LDR R1, =0x7161
        MUL R2, R0, R1
        BX LR
        AREA mydata, DATA, READWRITE
RESULT DCD 0
        END
```

4. Find the greatest common divisor

Exercise file: gcd of 2 32 bits.txt

Repeatedly subtracts the smaller value from the larger until the values match.

```
        PRESERVE8
        AREA RESET, DATA, READONLY
        EXPORT __Vectors
__Vectors
        DCD 0x10001000
        DCD Reset_Handler
        ALIGN
        AREA mycode, CODE, READONLY
        THUMB
        ENTRY
        EXPORT Reset_Handler
Reset_Handler
        MOV R0, #30
        MOV R1, #45
        CMP R0, #0
        BEQ FIRST_ZERO
        CMP R1, #0
        BEQ GCD_DONE
GCD_LOOP
        CMP R0, R1
        BEQ GCD_DONE
        BHI REDUCE_A
        SUB R1, R1, R0
        B GCD_LOOP
REDUCE_A
        SUB R0, R0, R1
        B GCD_LOOP
FIRST_ZERO
        MOV R0, R1
GCD_DONE
        LDR R2, =RESULT
        STR R0, [R2]
STOP    B STOP
        AREA mydata, DATA, READWRITE
RESULT DCD 0
        END
```

5. Find the least common multiple

Exercise file: lcm of 2 32 bits.txt

Uses a product and a subtraction-based GCD, followed by a division loop.

```
        PRESERVE8
        AREA RESET, DATA, READONLY
        EXPORT __Vectors
__Vectors
        DCD 0x10001000
        DCD Reset_Handler
        ALIGN
        AREA mycode, CODE, READONLY
        THUMB
        ENTRY
        EXPORT Reset_Handler
Reset_Handler
        MOV R0, #18
        MOV R1, #15
        MOV R4, #0
        CMP R0, #0
        BEQ SAVE_LCM
        CMP R1, #0
        BEQ SAVE_LCM
        MUL R3, R0, R1       ; product for the chosen small inputs
GCD_LOOP
        CMP R0, R1
        BEQ DIVIDE_SETUP
        BHI REDUCE_A
        SUB R1, R1, R0
        B GCD_LOOP
```

```
REDUCE_A
        SUB R0, R0, R1
        B GCD_LOOP
DIVIDE_SETUP
        MOV R5, R3           ; product / GCD by subtraction
DIVIDE_LOOP
        CMP R5, R0
        BLO SAVE_LCM
        SUB R5, R5, R0
        ADD R4, R4, #1
        B DIVIDE_LOOP
SAVE_LCM
        LDR R2, =RESULT
        STR R4, [R2]
STOP    B STOP
        AREA mydata, DATA, READWRITE
RESULT DCD 0
        END
```

6. Calculate a factorial

Exercise file: factorial.txt

Uses repeated multiplication; the zero and one cases both produce one.

```
        PRESERVE8
        AREA RESET, DATA, READONLY
        EXPORT __Vectors
__Vectors
        DCD 0x10001000
        DCD Reset_Handler
        ALIGN
        AREA mycode, CODE, READONLY
        THUMB
        ENTRY
        EXPORT Reset_Handler
Reset_Handler
        MOV R0, #5
        MOV R1, #1           ; 0! and 1! are both 1
        CMP R0, #1
        BLS FACT_DONE
FACT_LOOP
        MUL R2, R1, R0
        MOV R1, R2
        SUBS R0, R0, #1
        CMP R0, #1
        BHI FACT_LOOP
FACT_DONE
        LDR R3, =RESULT
        STR R1, [R3]
STOP    B STOP
        AREA mydata, DATA, READWRITE
RESULT DCD 0
        END
```

7. Generate the first ten Fibonacci numbers

Exercise file: first 10 fibnocci.txt

Updates consecutive Fibonacci values and stores them in memory.

```
        PRESERVE8
        AREA RESET, DATA, READONLY
        EXPORT __Vectors
__Vectors
        DCD 0x10001000
        DCD Reset_Handler
        ALIGN
        AREA mycode, CODE, READONLY
        THUMB
        ENTRY
        EXPORT Reset_Handler
Reset_Handler
        LDR R0, =RESULT
```

```
        MOV R1, #0
        MOV R2, #1
        MOV R3, #10
FIB_LOOP
        STR R1, [R0], #4
        ADD R4, R1, R2
        MOV R1, R2
        MOV R2, R4
        SUBS R3, R3, #1
        BNE FIB_LOOP
STOP    B STOP
        AREA mydata, DATA, READWRITE
RESULT SPACE 40
        END
```

8. Check whether values are even or odd

Exercise file: even or odd.txt

Uses bit tests and conditional execution to classify the supplied values.

```
        PRESERVE8
        AREA RESET, DATA, READONLY
        EXPORT __Vectors
__Vectors
        DCD 0x10001000
        DCD Reset_Handler
        ALIGN
        AREA mycode, CODE, READONLY
        THUMB
        ENTRY
        EXPORT Reset_Handler
Reset_Handler
        MOV R0, #4
        MOV R2, #0
        TST R0, #1
        BNE CHECK_ODD
        MOV R2, #1          ; first input is even
CHECK_ODD
        MOV R3, #5
        MOV R4, #0
        TST R3, #1
        BEQ SAVE_FLAGS
        MOV R4, #1          ; second input is odd
SAVE_FLAGS
        LDR R5, =RESULT
        STR R2, [R5]
        STR R4, [R5, #4]
STOP    B STOP
        AREA mydata, DATA, READWRITE
RESULT DCD 0, 0
        END
```

9. Check the sign of a value

Exercise file: check + or -.txt

Examines the high bit of a 32-bit value to distinguish positive and negative values.

```
        PRESERVE8
        AREA RESET, DATA, READONLY
        EXPORT __Vectors
__Vectors
        DCD 0x10001000
        DCD Reset_Handler
        ALIGN
        AREA mycode, CODE, READONLY
        THUMB
        ENTRY
        EXPORT Reset_Handler
Reset_Handler
        MOV R0, #4
        MOV R2, #0
        CMP R0, #0
```

```
        BLT CHECK_NEGATIVE
        MOV R2, #1          ; first input is nonnegative
CHECK_NEGATIVE
        LDR R3, =0x80000005
        MOV R4, #0
        CMP R3, #0
        BGE SAVE_FLAGS
        MOV R4, #1          ; second input is negative
SAVE_FLAGS
        LDR R5, =RESULT
        STR R2, [R5]
        STR R4, [R5, #4]
STOP    B STOP
        AREA mydata, DATA, READWRITE
RESULT DCD 0, 0
        END
```

10. Check whether two values are equal

Exercise file: equivalence of 2.txt

Shows equality checks using TEQ, XOR, and subtraction with flags.

```
        PRESERVE8
        AREA RESET, DATA, READONLY
        EXPORT __Vectors
__Vectors
        DCD 0x10001000
        DCD Reset_Handler
        ALIGN
        AREA mycode, CODE, READONLY
        THUMB
        ENTRY
        EXPORT Reset_Handler
Reset_Handler
        MOV R0, #2
        MOV R1, #2
        MOV R2, #0
        MOV R4, #0
        MOV R6, #0
        TEQ R0, R1
        BNE XOR_CHECK
        MOV R2, #1
XOR_CHECK
        EOR R3, R0, R1
        CMP R3, #0
        BNE SUB_CHECK
        MOV R4, #1
SUB_CHECK
        SUBS R5, R0, R1
        BNE SAVE_FLAGS
        MOV R6, #1
SAVE_FLAGS
        LDR R7, =RESULT
        STR R2, [R7]
        STR R4, [R7, #4]
        STR R6, [R7, #8]
STOP    B STOP
        AREA mydata, DATA, READWRITE
RESULT DCD 0, 0, 0
        END
```

11. Clear register R5

Exercise file: clear contents of r5.txt

Shows four ways to zero a register and stores each result.

```
        PRESERVE8
        AREA RESET, DATA, READONLY
        EXPORT __Vectors
__Vectors
        DCD 0x10001000
        DCD Reset_Handler
```

```
        ALIGN
        AREA mycode, CODE, READONLY
        THUMB
        ENTRY
        EXPORT Reset_Handler
Reset_Handler
        LDR R0, =RESULT
        LDR R5, =0x12345678
        AND R5, R5, #0
        STR R5, [R0]
        LDR R5, =0x12345678
        BIC R5, R5, R5
        STR R5, [R0, #4]
        LDR R5, =0x12345678
        SUB R5, R5, R5
        STR R5, [R0, #8]
        LDR R5, =0x12345678
        LSR R5, R5, #32
        STR R5, [R0, #12]
STOP    B STOP
        AREA mydata, DATA, READWRITE
RESULT DCD 0, 0, 0, 0
        END
```

12. Set selected bits in R4

Exercise file: set bits to 2,4,8 in r4.txt

Uses a bit mask and ORR to set selected bits without clearing the others.

```
        PRESERVE8
        AREA RESET, DATA, READONLY
        EXPORT __Vectors
__Vectors
        DCD 0x10001000
        DCD Reset_Handler
        ALIGN
        AREA mycode, CODE, READONLY
        THUMB
        ENTRY
        EXPORT Reset_Handler
Reset_Handler
        LDR R4, =0x00011111
        MOV R0, #0x8A       ; bit positions 2, 4, 8 (1-based)
        ORR R4, R4, R0
        LDR R1, =RESULT
        STR R4, [R1]
STOP    B STOP
        AREA mydata, DATA, READWRITE
RESULT DCD 0
        END
```

Lab 4

10 programs

Branching, Looping and Code Conversion

1. Convert hexadecimal to unpacked ASCII

Exercise file: convert hexa to unpack ascii.txt

Converts the high nibble and then the low nibble to ASCII characters.

```
        PRESERVE8
        AREA RESET, DATA, READONLY
        EXPORT __Vectors
__Vectors
        DCD 0x10001000
        DCD Reset_Handler
        ALIGN
        AREA mycode, CODE, READONLY
        THUMB
        ENTRY
        EXPORT Reset_Handler
```

```
Reset_Handler
        LDR R0, =NUM
        LDR R3, =RESULT
        LDRB R1, [R0]
        MOV R2, R1, LSR #4  ; upper hexadecimal digit
        CMP R2, #9
        BLS HIGH_DIGIT
        ADD R2, R2, #7
HIGH_DIGIT
        ADD R2, R2, #0x30
        STRB R2, [R3]
        AND R4, R1, #0x0F   ; lower hexadecimal digit
        CMP R4, #9
        BLS LOW_DIGIT
        ADD R4, R4, #7
LOW_DIGIT
        ADD R4, R4, #0x30
        STRB R4, [R3, #1]
STOP    B STOP
NUM     DCB 0x3A
        ALIGN
        AREA mydata, DATA, READWRITE
RESULT DCB 0, 0
        END
```

2. Convert lowercase text to uppercase

Exercise file: lower to upper.txt

Converts lowercase ASCII letters while copying the remaining characters unchanged.

```
        PRESERVE8
        AREA RESET, DATA, READONLY
        EXPORT __Vectors
__Vectors
        DCD 0x10001000
        DCD Reset_Handler
        ALIGN
        AREA mycode, CODE, READONLY
        THUMB
        ENTRY
        EXPORT Reset_Handler
Reset_Handler
        LDR R0, =TEXT
        LDR R2, =OUTPUT
NEXT_CHAR
        LDRB R1, [R0], #1
        CMP R1, #0
        BEQ TERMINATE
        CMP R1, #0x61       ; 'a'
        BLO COPY_CHAR
        CMP R1, #0x7A       ; 'z'
        BHI COPY_CHAR
        SUB R1, R1, #0x20
COPY_CHAR
        STRB R1, [R2], #1
        B NEXT_CHAR
TERMINATE
        STRB R1, [R2]      ; zero terminator
STOP    B STOP
TEXT    DCB "word", 0
        ALIGN
        AREA mydata, DATA, READWRITE
OUTPUT SPACE 5
        END
```

3. Generate the first ten even numbers

Exercise file: generate first 10 even.txt

Uses a loop to store ten consecutive even values in memory.

```
        PRESERVE8
        AREA RESET, DATA, READONLY
        EXPORT __Vectors
__Vectors
        DCD 0x10001000
        DCD Reset_Handler
        ALIGN
        AREA mycode, CODE, READONLY
        THUMB
        ENTRY
        EXPORT Reset_Handler
Reset_Handler
        MOV R0, #0
        MOV R1, #10
        LDR R2, =RESULT
EVEN_LOOP
        STR R0, [R2], #4
        ADD R0, R0, #2
        SUBS R1, R1, #1
        BNE EVEN_LOOP
STOP    B STOP
        AREA mydata, DATA, READWRITE
RESULT SPACE 40
        END
```

4. Generate the first ten odd numbers

Exercise file: generate first 10 odd numbers.txt

Uses a loop to store ten consecutive odd values in memory.

```
        PRESERVE8
        AREA RESET, DATA, READONLY
        EXPORT __Vectors
__Vectors
        DCD 0x10001000
        DCD Reset_Handler
        ALIGN
        AREA mycode, CODE, READONLY
        THUMB
        ENTRY
        EXPORT Reset_Handler
Reset_Handler
        MOV R0, #1
        MOV R1, #10
        LDR R2, =RESULT
ODD_LOOP
        STR R0, [R2], #4
        ADD R0, R0, #2
        SUBS R1, R1, #1
        BNE ODD_LOOP
STOP    B STOP
        AREA mydata, DATA, READWRITE
RESULT SPACE 40
        END
```

5. Find the greater of two numbers

Exercise file: greatest of 2 32 bits.txt

Compares two register values and selects the greater one.

```
        PRESERVE8
        AREA RESET, DATA, READONLY
        EXPORT __Vectors
__Vectors
        DCD 0x10001000
        DCD Reset_Handler
        ALIGN
        AREA mycode, CODE, READONLY
```

```
        THUMB
        ENTRY
        EXPORT Reset_Handler
Reset_Handler
        MOV R0, #5
        MOV R1, #6
        CMP R0, R1
        BHS FIRST_GREATER
        MOV R2, R1
        B STORE_MAX
FIRST_GREATER
        MOV R2, R0
STORE_MAX
        LDR R3, =RESULT
        STR R2, [R3]
STOP    B STOP
        AREA mydata, DATA, READWRITE
RESULT DCD 0
        END
```

6. Find the greatest of five register values

Exercise file: greatest of 5 32 bits.txt

Compares five values held in registers, retaining the largest.

```
        PRESERVE8
        AREA RESET, DATA, READONLY
        EXPORT __Vectors
__Vectors
        DCD 0x10001000
        DCD Reset_Handler
        ALIGN
        AREA mycode, CODE, READONLY
        THUMB
        ENTRY
        EXPORT Reset_Handler
Reset_Handler
        MOV R0, #2
        MOV R1, #3
        MOV R2, #1
        MOV R3, #6
        MOV R4, #5
        MOV R5, R0
        CMP R1, R5
        BLS CHECK_2
        MOV R5, R1
CHECK_2
        CMP R2, R5
        BLS CHECK_3
        MOV R5, R2
CHECK_3
        CMP R3, R5
        BLS CHECK_4
        MOV R5, R3
CHECK_4
        CMP R4, R5
        BLS STORE_MAX
        MOV R5, R4
STORE_MAX
        LDR R6, =RESULT
        STR R5, [R6]
STOP    B STOP
        AREA mydata, DATA, READWRITE
RESULT DCD 0
        END
```

7. Find the greatest of five values using a loop

Exercise file: greatest of 5 32 bits with loop.txt

Scans five memory values and updates the current maximum.

```
        PRESERVE8
        AREA RESET, DATA, READONLY
        EXPORT __Vectors
__Vectors
        DCD 0x10001000
        DCD Reset_Handler
        ALIGN
        AREA mycode, CODE, READONLY
        THUMB
        ENTRY
        EXPORT Reset_Handler
Reset_Handler
        LDR R0, =NUMBERS
        LDR R3, [R0], #4   ; first value is initial maximum
        MOV R2, #4         ; four values remain
MAX_LOOP
        LDR R4, [R0], #4
        CMP R4, R3
        BLS NEXT_VALUE
        MOV R3, R4
NEXT_VALUE
        SUBS R2, R2, #1
        BNE MAX_LOOP
        LDR R5, =RESULT
        STR R3, [R5]
STOP    B STOP
NUMBERS DCD 2, 3, 1, 6, 5
        AREA mydata, DATA, READWRITE
RESULT DCD 0
        END
```

8. Reverse a string

Exercise file: reverse a string.txt

Finds the last character and copies the string backward into an output buffer.

```
        PRESERVE8
        AREA RESET, DATA, READONLY
        EXPORT __Vectors
__Vectors
        DCD 0x10001000
        DCD Reset_Handler
        ALIGN
        AREA mycode, CODE, READONLY
        THUMB
        ENTRY
        EXPORT Reset_Handler
Reset_Handler
        LDR R0, =TEXT
        LDR R2, =OUTPUT
        MOV R3, R0
FIND_END
        LDRB R1, [R0], #1
        CMP R1, #0
        BNE FIND_END
        SUB R0, R0, #2     ; last character
REVERSE_LOOP
        CMP R0, R3
        BLO TERMINATE
        LDRB R1, [R0], #-1
        STRB R1, [R2], #1
        B REVERSE_LOOP
TERMINATE
        MOV R1, #0
        STRB R1, [R2]
STOP    B STOP
TEXT    DCB "WORD", 0
        ALIGN
```

```
        AREA mydata, DATA, READWRITE
OUTPUT SPACE 5
        END
```

9. Check a string for a palindrome

Exercise file: palindrome.txt

Compares opposite characters and stores one for a palindrome, zero otherwise.

```
        PRESERVE8
        AREA RESET, DATA, READONLY
        EXPORT __Vectors
__Vectors
        DCD 0x10001000
        DCD Reset_Handler
        ALIGN
        AREA mycode, CODE, READONLY
        THUMB
        ENTRY
        EXPORT Reset_Handler
Reset_Handler
        LDR R0, =TEXT
        MOV R1, R0
        MOV R2, #0
FIND_END
        LDRB R3, [R0], #1
        CMP R3, #0
        BEQ SETUP_COMPARE
        ADD R2, R2, #1
        B FIND_END
SETUP_COMPARE
        MOV R5, #0         ; false unless all pairs match
        CMP R2, #1
        BLS PAL_TRUE
        SUB R0, R0, #2     ; point to final character
PAL_LOOP
        CMP R1, R0
        BHS PAL_TRUE
        LDRB R3, [R1], #1
        LDRB R4, [R0], #-1
        CMP R3, R4
        BNE SAVE_RESULT
        B PAL_LOOP
PAL_TRUE
        MOV R5, #1
SAVE_RESULT
        LDR R6, =RESULT
        STR R5, [R6]
STOP    B STOP
TEXT    DCB "LEVEL", 0
        ALIGN
        AREA mydata, DATA, READWRITE
RESULT DCD 0
        END
```

10. Count spaces in a string

Exercise file: number od blank spaces.txt

Scans a character string and counts the space characters.

```
        PRESERVE8
        AREA RESET, DATA, READONLY
        EXPORT __Vectors
__Vectors
        DCD 0x10001000
        DCD Reset_Handler
        ALIGN
        AREA mycode, CODE, READONLY
        THUMB
        ENTRY
        EXPORT Reset_Handler
Reset_Handler
```

```
        LDR R0, =TEXT
        MOV R1, #0
COUNT_LOOP
        LDRB R2, [R0], #1
        CMP R2, #0
        BEQ SAVE_COUNT
        CMP R2, #0x20
        BNE COUNT_LOOP
        ADD R1, R1, #1
        B COUNT_LOOP
SAVE_COUNT
        LDR R3, =RESULT
        STR R1, [R3]
STOP    B STOP
TEXT    DCB "A A A A", 0
        ALIGN
        AREA mydata, DATA, READWRITE
RESULT DCD 0
        END
```

Lab 5

2 programs

Sorting and Searching

1. Sort ten values using bubble sort

Exercise file: bubble sort.txt

Copies ten words to a 40-byte output buffer and sorts them in ascending order.

```
        PRESERVE8
        AREA RESET, DATA, READONLY
        EXPORT __Vectors
__Vectors
        DCD 0x10001000
        DCD Reset_Handler
        ALIGN
        AREA mycode, CODE, READONLY
        THUMB
        ENTRY
        EXPORT Reset_Handler
Reset_Handler
        LDR R0, =LIST
        LDR R1, =RESULT
        MOV R2, #10
COPY_LOOP
        LDR R3, [R0], #4
        STR R3, [R1], #4
        SUBS R2, R2, #1
        BNE COPY_LOOP

        MOV R9, #9         ; number of passes
OUTER_LOOP
        LDR R5, =RESULT
        MOV R4, R9         ; comparisons in this pass
INNER_LOOP
        LDR R6, [R5]
        LDR R7, [R5, #4]
        CMP R6, R7
        BLS NO_SWAP
        STR R7, [R5]
        STR R6, [R5, #4]
NO_SWAP
        ADD R5, R5, #4
        SUBS R4, R4, #1
        BNE INNER_LOOP
        SUBS R9, R9, #1
        BNE OUTER_LOOP
STOP    B STOP
LIST    DCD 0x10, 0x05, 0x33, 0x24, 0x56
        DCD 0x77, 0x21, 0x04, 0x87, 0x01
        AREA mydata, DATA, READWRITE
```

```
RESULT SPACE 40
        END
```

2. Search for a character in a string

Exercise file: search given char in string.txt

Finds the letter O in WORD and stores a found flag and zero-based index.

```
        PRESERVE8
        AREA RESET, DATA, READONLY
        EXPORT __Vectors
__Vectors
        DCD 0x10001000
        DCD Reset_Handler
        ALIGN
        AREA mycode, CODE, READONLY
        THUMB
        ENTRY
        EXPORT Reset_Handler
Reset_Handler
        LDR R0, =TEXT
        MOV R4, #0x4F      ; 'O' is present in "WORD"
        MOV R5, #0         ; zero-based index
        MOV R6, #0         ; found flag
SEARCH_LOOP
        LDRB R1, [R0], #1
        CMP R1, #0
        BEQ SAVE_RESULT
        CMP R1, R4
        BEQ FOUND
        ADD R5, R5, #1
        B SEARCH_LOOP
FOUND
        MOV R6, #1
SAVE_RESULT
        LDR R7, =RESULT
        STR R6, [R7]
        STR R5, [R7, #4]  ; index is useful when found=1
STOP    B STOP
TEXT    DCB "WORD", 0
        ALIGN
        AREA mydata, DATA, READWRITE
RESULT DCD 0, 0
        END
```

Hardware

5 programs

LPC17xx Interfacing Programs

1. LED on and off

Uploaded file: Ledonoff.c

Toggles Port 2 output pins with a delay between states.

```
 #include <lpc17xx.h>

void delay_ms(unsigned int ms)
{
    unsigned int i, j;

    for(i = 0; i < ms; i++)
        for(j = 0; j < 50000; j++);
}

/* start the main program */
int main()
{
    SystemInit();   // Clock and PLL configuration
    LPC_PINCON->PINSEL4 = 0x000000;   // Configure the PORT2 Pins as GPIO
    LPC_GPIO2->FIODIR = 0xFFFFFFFF;   // Configure the PORT2 pins as OUTPUT
```

```
    while(1)
    {
        LPC_GPIO2->FIOSET = 0xFFFFFFFF;   // Make all the Port pins high
        delay_ms(100);

        LPC_GPIO2->FIOCLR = 0xFFFFFFFF;   // Make all the Port pins low
        delay_ms(100);
    }
}
```

2. Seven-segment display

Uploaded file: Seven_seg.c

Cycles through hexadecimal digits using a lookup table and GPIO outputs.

```
#include <LPC17xx.h>
unsigned int delay, count=0, Switchcount=0,j;

unsigned int Disp[16]={0x000003f0, 0x00000060, 0x000005b0, 0x000004f0, 0x00000660,0x000006d0,
                       0x000007d0, 0x00000070, 0x000007f0, 0x000006f0, 0x00000770,0x000007c0,
                       0x00000390, 0x000005e0, 0x00000790, 0x00000710 };

#define ALLDISP  0x00180000
#define DATAPORT 0x00000ff0
int main (void)
{
    LPC_PINCON->PINSEL0 = 0x00000000;
    LPC_PINCON->PINSEL1 = 0x00000000;
    LPC_GPIO0->FIODIR = 0x00180ff0;

    while(1)
    {
        LPC_GPIO0->FIOSET |= ALLDISP;
        LPC_GPIO0->FIOCLR = 0x00000ff0;
        LPC_GPIO0->FIOSET = Disp[Switchcount];

             for(j=0;j<3;j++)
            for(delay=0;delay<30000;delay++);

                Switchcount++;
                if(Switchcount == 0x10)

                   Switchcount = 0;
                   LPC_GPIO0->FIOCLR  = 0x00180ff0;
                }
    }
```

3. Relay and buzzer control

Uploaded file: Relay.c

Reads a switch input to control relay and buzzer output pins.

```
#include <LPC17xx.H>
int main(void)
{
    LPC_PINCON->PINSEL1 = 0x00000000;                                  //P0.24,P0.25 GPIO
    LPC_GPIO0->FIODIR = 0x03000000;                                    //P0.24 configured output for buzze
    r,P0.25 configured output for Relay/Led
   while(1)
   {
                if(!(LPC_GPIO2->FIOPIN & 0x00000800))           //Is GP_SW(SW4) is pressed??
                {
                     LPC_GPIO0->FIOSET = 0x03000000;                //relay on
                }
                else
                {
                     LPC_GPIO0->FIOCLR = 0x03000000;                //relay off
                }
     }
}                                                                                                  //e
    nd int main(void)
```

4. ADC input with LCD output

Uploaded file: INT_ADC.c

Reads an ADC value and displays its raw value and calculated voltage.

```
#include<LPC17xx.h>
#include "lcd.h"
#include<stdio.h>
#define Ref_Vtg     3.300
#define Full_Scale  0xFFF               //12 bit ADC

int main(void)
{
    unsigned int adc_temp;
    unsigned int i;
    float in_vtg;
    unsigned char vtg[7],dval[7], blank[]="   ";
    unsigned char Msg3[11] = {"ANALOG IP:"};
    unsigned char Msg4[12] = {"ADC OUTPUT:"};

   lcd_init();
   LPC_PINCON->PINSEL3 |= 0xC0000000;
     //P1.31 as AD0.5
     LPC_SC->PCONP |= (1<<12);
             //enable the peripheral ADC

    temp1 = 0x80;
    lcd_com();
    delay_lcd(800);
    lcd_puts(&Msg3[0]);

    temp1 = 0xC0;
    lcd_com();
    delay_lcd(800);
    lcd_puts(&Msg4[0]);

    while(1)
    {
        LPC_ADC->ADCR = (1<<5)|(1<<21)|(1<<24);
     //0x01200001;//ADC0.5, start conversion and operational
        for(i=0;i<2000;i++);
                        //delay for conversion
        while((adc_temp = LPC_ADC->ADGDR) == 0x80000000);                                       //wait
    till 'done' bit is 1, indicates conversion complete
        adc_temp = LPC_ADC->ADGDR;
        adc_temp >>= 4;
        adc_temp &= 0x00000FFF;
                    //12 bit ADC
        in_vtg = (((float)adc_temp * (float)Ref_Vtg))/((float)Full_Scale);  //calculating input analog
    voltage
        sprintf(vtg,"%3.2fV",in_vtg);
                //convert the readings into string to display on LCD
        sprintf(dval,"%x",adc_temp);
        for(i=0;i<2000;i++);

        temp1 = 0x8A;
        lcd_com();
        delay_lcd(800);
        lcd_puts(&vtg[0]);

        temp1 = 0xCB;
        lcd_com();
        lcd_puts(&blank[0]);

        temp1 = 0xCB;
        lcd_com();
        delay_lcd(800);
        lcd_puts(&dval[0]);

        for(i=0;i<200000;i++);
        for(i=0;i<7;i++)
        vtg[i] = dval[i] = 0x00;
        adc_temp = 0;
```

```
        in_vtg = 0;
    }
}
```

5. Stepper motor rotation

Uploaded file: stpper.c

Steps through clockwise and anticlockwise GPIO output sequences.

```
#include <LPC17xx.H>
void clock_wise(void);
void anti_clock_wise(void);
unsigned long int var1,var2;
unsigned int i=0,j=0,k=0;

int main(void)
{
    LPC_PINCON->PINSEL4 = 0x00000000;       //P2.0 to P2.3 GPIO
    LPC_GPIO2->FIODIR = 0x0000000F;         //P2.0 to P2.3 output
    while(1)
    {
        for(j=0;j<30;j++)                       //50 times in Clock wise Rotation
            clock_wise();

        for(k=0;k<50000;k++);                   //Delay to show  anti_clock Rotation

        for(j=0;j<30;j++)                       //50 times in  Anti Clock wise Rotation
            anti_clock_wise();

        for(k=0;k<50000;k++) ;                  //Delay to show clock Rotation
    }                                                               //End of while(1)
}                                                                   //End of main
void clock_wise(void)
{
        var1 = 0x00000001;                      //For Clockwise
    for(i=0;i<=3;i++)                       //for A B C D Stepping
    {
       LPC_GPIO2->FIOCLR =  0X0000000F;
       LPC_GPIO2->FIOSET =  var1;
         var1 = var1<<1;                            //For Clockwise
     for(k=0;k<15000;k++) ;                     //for step speed variation
  }
}
void anti_clock_wise(void)
{
        var1 = 0x0000008;                           //For Anticlockwise
    for(i=0;i<=3;i++)                           //for A B C D Stepping
    {
        LPC_GPIO2->FIOCLR =  0X0000000F;
            LPC_GPIO2->FIOSET =  var1;
            var1 = var1>>1;                             //For Anticlockwise
      for(k=0;k<15000;k++);                     //for step speed variation
    }
}
```